NAME: Akshata Katkar
PRN: 124A8042
CLASS/DIV: SE-AIDS(G2)
DATE OF SUBMISSION: 30/01/2026

EXPERIMENT NO.: 03

Aim : Linux Processes

Theory:

Linux Process:

A Linux process is an instance of a program in execution. It is created when a program runs and is assigned system resources such as CPU, memory, and files.

Linux Process States:

A process in Linux moves through different states during its lifetime. It may be running, sleeping while waiting for an event, stopped by a signal, or become a zombie after execution completes if the parent does not collect its status. Once cleaned up, the process is fully removed from the system.

System Calls in Linux:

System calls are used by user programs to interact with the Linux kernel. They provide controlled access to system resources and services such as process creation and management. Common process-related system calls include fork(), getpid(), getppid(), and wait().

fork():

The fork() system call creates a new process called a child process, which is a copy of the parent process. After fork(), both parent and child execute the same code, with the child receiving a return value of zero and the parent receiving the child's process ID.

getpid():

The getpid() system call returns the unique process ID of the calling process. It is mainly used for process identification, debugging, and monitoring.

getppid():

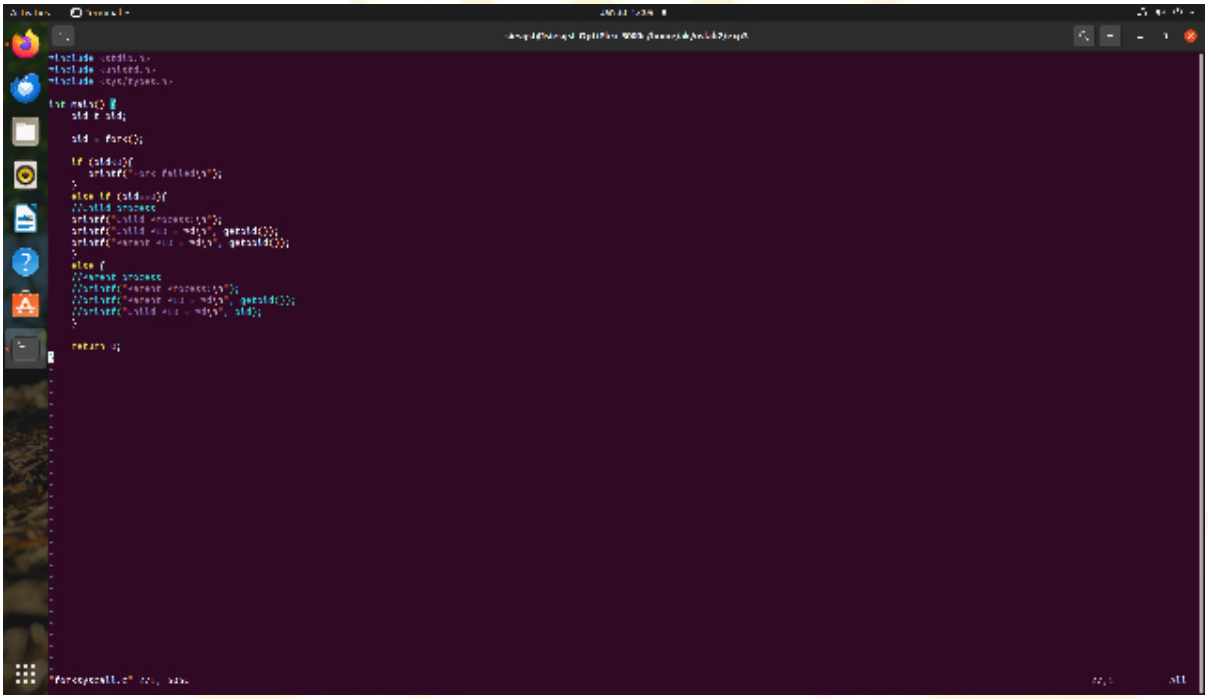
The getppid() system call returns the process ID of the parent of the calling process. It helps identify the parent–child relationship between processes.

wait():

The wait() system call makes the parent process pause execution until its child process finishes. It collects the child's exit status and prevents zombie processes.

A. Create a child process in Linux using the fork system call. From the child process, obtain the process ID of both child and parent by using getpid and getppid system calls.

Code:



Output:

```
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ vi forksyscall.c
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ gcc forksyscall.c -o forksyscall
[1]-  Done                    ./fork_sleepsyscall
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ ./forksyscall
Parent Process:
Parent PID = 3702
Child PID = 3703
Child Process:
Child PID = 3703
Parent PID = 1421
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ ./forksyscall
Parent Process:
Parent PID = 3706
Child PID = 3707
Child Process:
Child PID = 3707
Parent PID = 1421
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ ./forksyscall
Parent Process:
Parent PID = 3711
Child PID = 3712
Child Process:
Child PID = 3712
Parent PID = 1421
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$
```

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid<0){
       printf("Fork failed\n");
    }
    else if (pid==0){
    //Child process
    printf("Child Process:\n");
    printf("Child PID = %d\n", getpid());
    printf("Parent PID = %d\n", getppid());
    sleep(30);   //child stays alive
    }
    else {
    //Parent process
    printf("Parent Process:\n");
    printf("Parent PID = %d\n", getpid());
    printf("Child PID = %d\n", pid);
    sleep(300);  //parent stays alive

    }

    return 0;
}
```

Output:

```
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ ./fork_sleepsyscall &
[1] 3535
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ Parent Process:
Parent PID = 3535
Child PID = 3536
Child Process:
Child PID = 3536
Parent PID = 3535
ps -ef | grep fork_sleepsyscall
siesgst     3535    2676  0 12:33 pts/0    00:00:00 ./fork_sleepsyscall
siesgst     3536    3535  0 12:33 pts/0    00:00:00 ./fork_sleepsyscall
siesgst     3547    2676  0 12:33 pts/0    00:00:00 grep --color=auto fork_sleepsyscall
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ ./fork_sleepsyscall &
[2] 3554
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ Parent Process:
Parent PID = 3554
Child PID = 3555
Child Process:
Child PID = 3555
Parent PID = 3554
ps -ef | grep fork_sleepsyscall
siesgst     3535    2676  0 12:33 pts/0    00:00:00 ./fork_sleepsyscall
siesgst     3554    2676  0 12:35 pts/0    00:00:00 ./fork_sleepsyscall
siesgst     3555    3554  0 12:35 pts/0    00:00:00 ./fork_sleepsyscall
siesgst     3557    2676  0 12:35 pts/0    00:00:00 grep --color=auto fork_sleepsyscall
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$
```

B. Explore wait and waitpid before termination of the process.

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();


    if (pid==0){
    //Child process
    printf("Child Process:\n");
    printf("Child PID = %d\n", getpid());
    printf("Parent PID = %d\n", getppid());
    sleep(10);
    printf("Child existing\n");
    }
    else {
    //Parent process
    printf("Parent Process:\n");
    printf("Parent PID = %d\n", getpid());
    printf("Waiting for child to finish...\n");

    wait(&status);

    printf("Child finished. Parent existing. \n");
    }

    return 0;
}
```

Output:

```
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ vi fork_wait_syscall.c
[2]+  Done                    ./fork_sleepsyscall
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ gcc fork_wait_syscall.c -o fork_wait_syscall
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ ./fork_wait_syscall
Parent Process:
Parent PID = 3836
Waiting for child to finish...
Child Process:
Child PID = 3837
Parent PID = 3836
^C
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ ./fork_wait_syscall &
[1] 3838
siesgst@siesgst-OptiPlex-3000:/home/ak/oslab2/exp3$ Parent Process:
Parent PID = 3838
Waiting for child to finish...
Child Process:
Child PID = 3839
Parent PID = 3838
Child existing
Child finished. Parent existing.
^C
[1]+  Done                    ./fork_wait_syscall
```